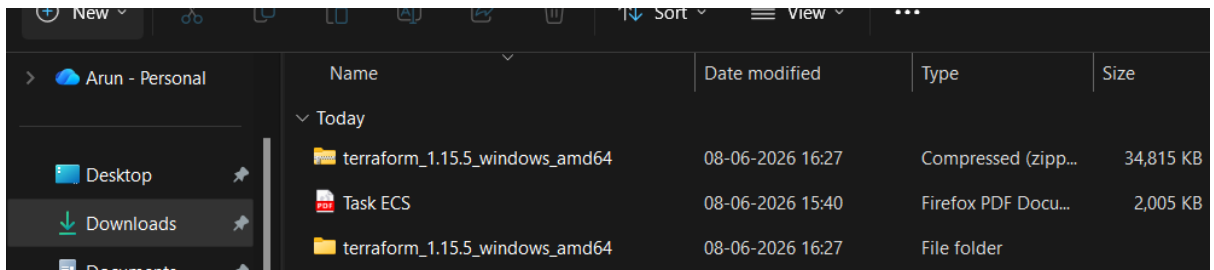


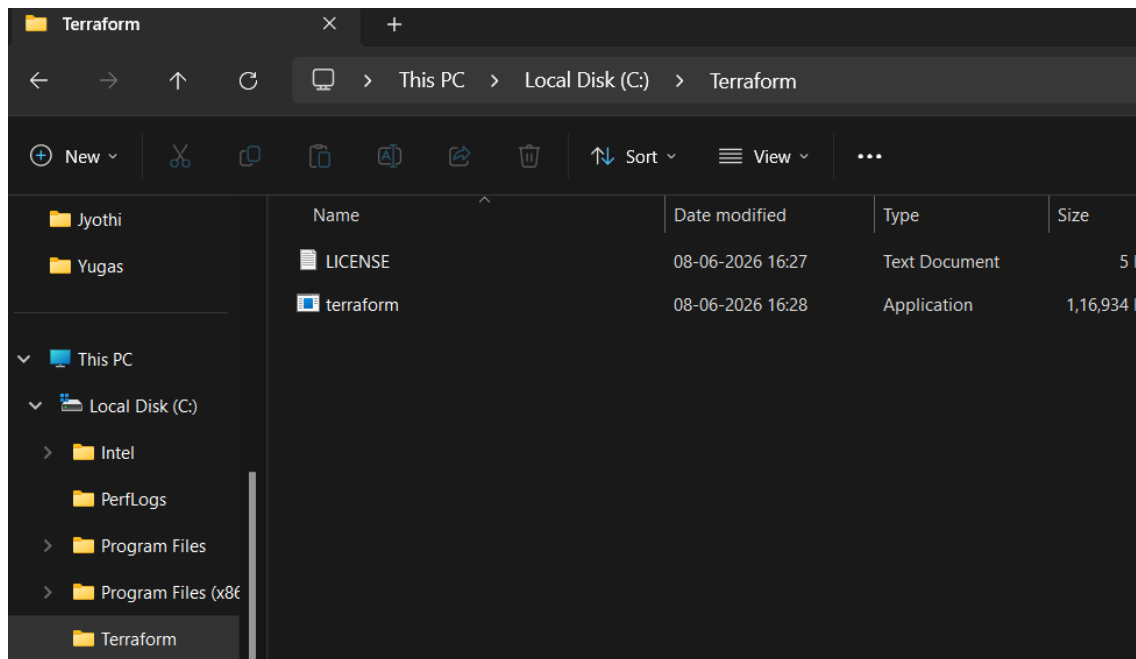
Terraform-01

1) Install Terraform on your PC.

- Install Terraform from Host Crop Website select latest Version and extract it

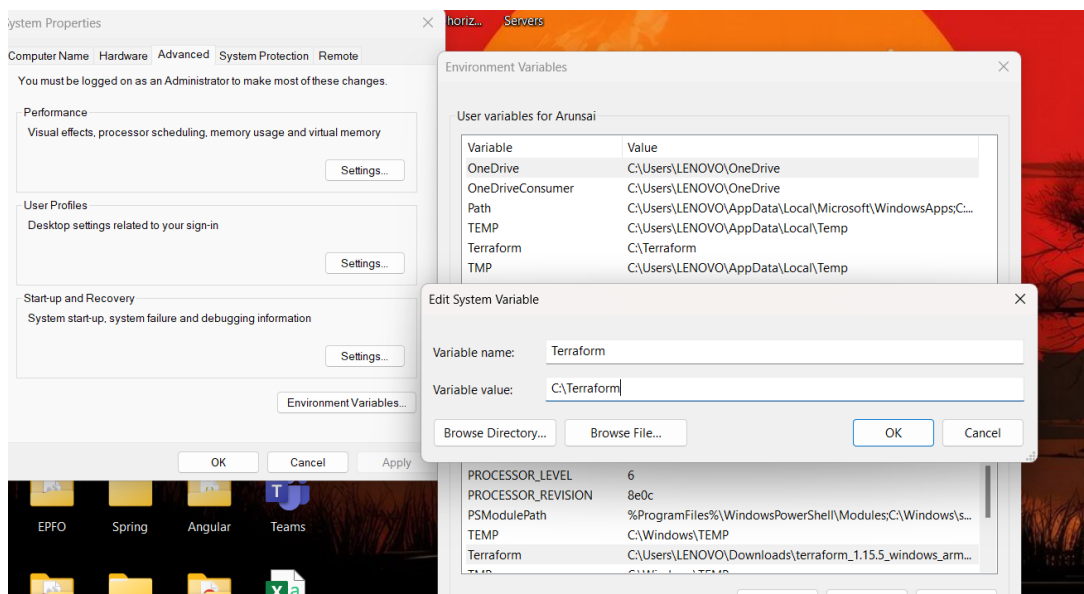
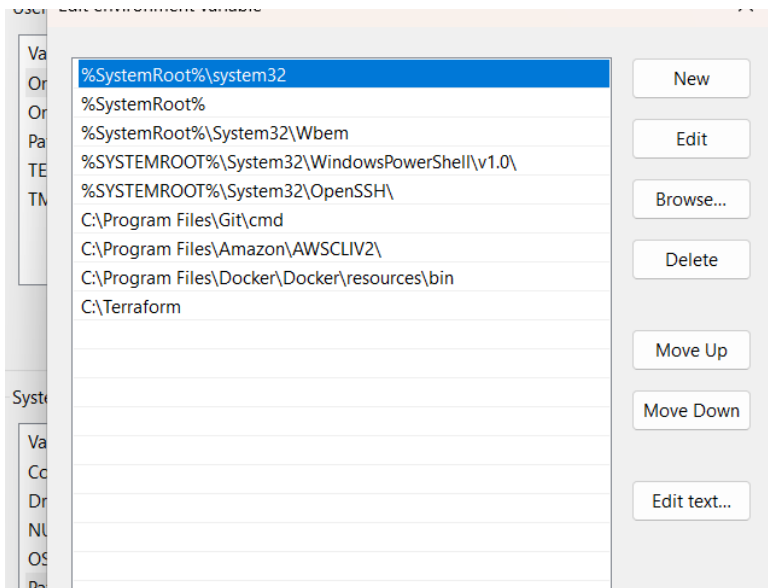


- Created Terraform Folder in Windows C Drive and paste extract Terraform application



- Now open Environment Variables
- Under **System Variables**
- Select **Path**
- Click **Edit**
- Click **New**
- ADD

C:\Terraform



- Now open in Windows C Drive location open terminal and check Terraform Version

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

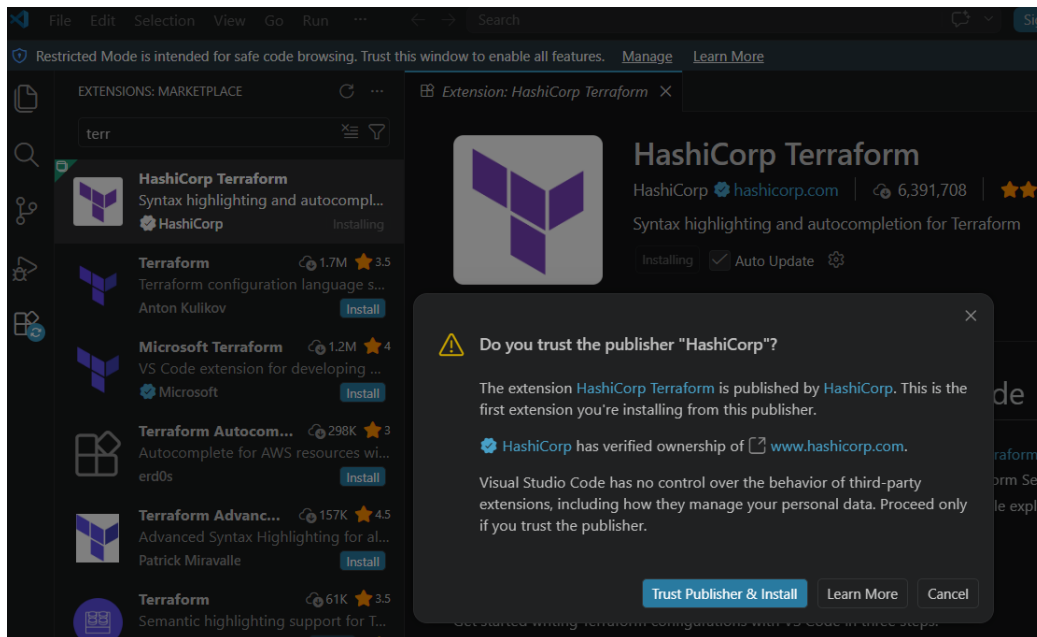
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Terraform> .\terraform.exe -version
Terraform v1.15.5
on windows_amd64
PS C:\Terraform>
```

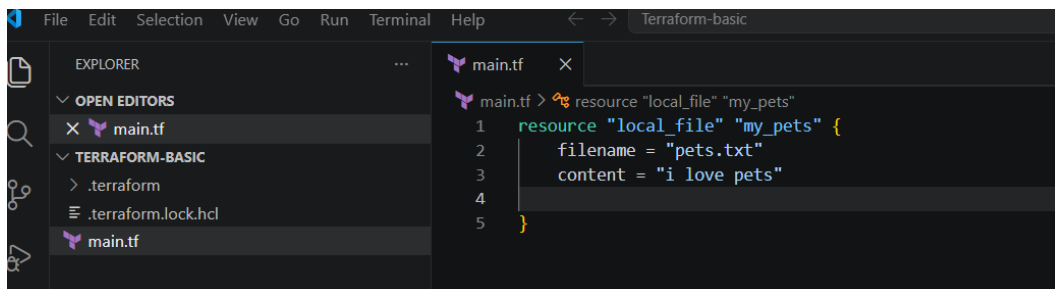
- PATH is configured correctly

2) Execute all the templates shown in video.

- Open Vs code and install Hashi Corp Terraform



- In Desktop created terraform folder and open it create a **main.tf** file write a resource script



- Now use command **terraform init**

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/local...
- Installing hashicorp/local v2.9.0...
- Installed hashicorp/local v2.9.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\LENOVO\Desktop\Terraform-basic> 
```

- Now with **terraform plan** it adds file

commands will detect it and remind you to do so if necessary.

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform plan
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- + create

Terraform will perform the following actions:

```
# local_file.my_pets will be created
+ resource "local_file" "my_pets" {
  + content           = "i love pets"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
```

```
  + file_permission    = "0777"
  + filename            = "pets.txt"
  + id                  = (known after apply)
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

```
PS C:\Users\LENOVO\Desktop\Terraform-basic>
```

- Now with **terraform apply** it creates file

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- + create

Terraform will perform the following actions:

```
# local_file.my_pets will be created
+ resource "local_file" "my_pets" {
  + content           = "i love pets"
  + content_base64sha256 = (known after apply)
```

```
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

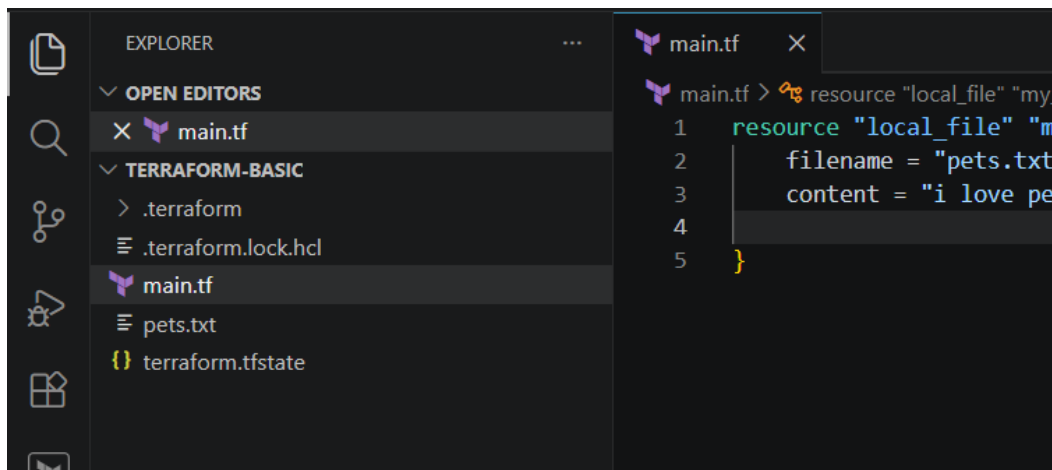
local_file.my_pets: Creating...

local_file.my_pets: Creation complete after 0s [id=24899fdea020fed9c33448b53811e17d15bb4f25]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

```
PS C:\Users\LENOVO\Desktop\Terraform-basic>
```

➤ Now we can see **terraform.tfstate**



If we run terraform apply again without doing any changes in configuration file

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
local_file.my_pets: Refreshing state... [id=24899fdea020fed9c33448b53811e17d15bb4f25]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences,
so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform-basic>
```

3) Note down below points, Terraform Init Terraform Plan Terraform Apply Terraform Provider.

1. Terraform Init

Command **terraform init**

Purpose:

- Initializes the Terraform working directory.
- Downloads required provider plugins.
- Creates .terraform folder.
- Creates .terraform.lock.hcl file

2. Terraform Plan

Command **terraform plan**

Purpose:

- Shows what Terraform will create, modify, or destroy.
- Does not make any changes.
- Used to verify infrastructure before deployment.
- Meaning Terraform planned to create one file.

3. Terraform Apply

Command **terraform apply**

Purpose:

- Executes the plan.
- Creates actual resources.

Meaning Terraform compared the current state with the configuration and found everything already exists.

4. Terraform Provider

It is a complete package of Api calls to communicate with our resources
an Acts as the bridge between Terraform and the target infrastructure platform

4). Integrate a sample Terraform template in Jenkins.

- Connect With Jenkins server start Jenkins and check running or not

```
root@ip-172-31-22-87:~# systemctl start jenkins
root@ip-172-31-22-87:~# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-06-09 06:07:44 UTC; 1min 4s ago
     Main PID: 522 (java)
        Tasks: 40 (limit: 1013)
      Memory: 380.0M (peak: 461.3M)
         CPU: 28.980s
    CGroup: /system.slice/jenkins.service
            └─522 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --
Jun 09 06:07:44 ip-172-31-22-87 jenkins[522]: 2026-06-09 06:07:44.590+0000 [id=30]      IN
Jun 09 06:07:44 ip-172-31-22-87 jenkins[522]: 2026-06-09 06:07:44.604+0000 [id=31]      IN
```

- Install Terraform in Jenkins Server

```
root@ip-172-31-22-87:~# terraform -version
Command 'terraform' not found, but can be installed with:
snap install terraform
root@ip-172-31-22-87:~# sudo apt update
sudo apt install -y gnupg software-properties-common curl

curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/h
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.ha
shicorp.list
sudo apt update
sudo apt install terraform -y
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Ign:4 https://pkg.jenkins.io/debian-stable binary/ InRelease
Get:5 https://pkg.jenkins.io/debian-stable binary/ Release [2044 B]
```

- Verify Terraform version

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-172-31-22-87:~# terraform -version
Terraform v1.15.5
on linux_amd64
root@ip-172-31-22-87:~#
```

➤ Create vi main.tf for Simple Terraform

```
root@ip-172-31-22-87:~# cd /var/lib/jenkins/workspace/"Simple Terraform"
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Simple Terraform# vi main.tf
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Simple Terraform# ls
main.tf
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Simple Terraform#
```

➤ Write a script for it

```
resource "local_file" "my_pets" {
  filename = "pets.txt"
  content  = "i love pets"
}
```

➤ Open Jenkins In Browser and create a Job for pipeline

➤ In Pipeline written Declarative pipeline

Pipeline script

Script ?

```
10     }
11
12     stage('Terraform Plan') {
13         steps {
14             sh 'terraform plan'
15         }
16     }
17
18     stage('Terraform Apply') {
19         steps {
20             sh 'terraform apply -auto-approve'
21         }
22     }
23 }
24 }
```

➤ Successfully executed



Jenkins

/ Simple Terraform

/ #2

```
[32m+[0m[0m content_base64sha256 = (known after apply)
[32m+[0m[0m content_base64sha512 = (known after apply)
[32m+[0m[0m content_md5           = (known after apply)
[32m+[0m[0m content_sha1           = (known after apply)
[32m+[0m[0m content_sha256         = (known after apply)
[32m+[0m[0m content_sha512         = (known after apply)
[32m+[0m[0m directory_permission = "0777"
[32m+[0m[0m file_permission      = "0777"
[32m+[0m[0m filename             = "pets.txt"
[32m+[0m[0m id                   = (known after apply)
}
```

```
[1mPlan:[0m [0m1 to add, 0 to change, 0 to destroy.
[0m[1mlocal_file.my_pets: Creating...[0m[0m
[0m[1mlocal_file.my_pets: Creation complete after 0s [id=24899fde
[0m[1m[32m
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.[0m
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```